

POO C# Avancée

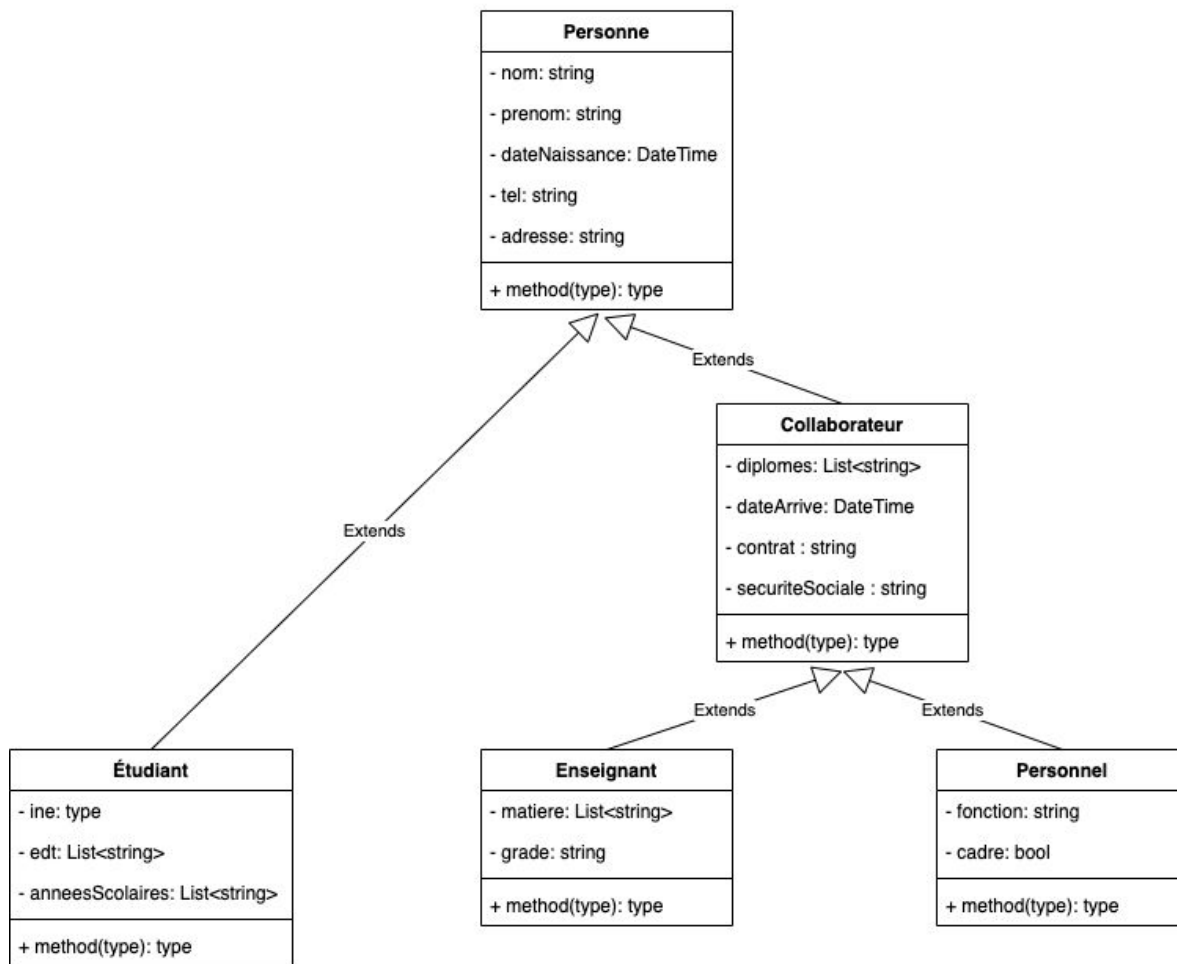
SIO2 - Antoine MATHAT

Sommaire

- Héritage et abstraction
- Classe et méthode statique
- Interface
- Redéfinition de méthode (Polymorphisme)

Héritage et abstraction

Héritage



Héritage

```
Personne Nicolas = new Collaborateur("Nicolas", "Pascal", new DateTime(2020, 12, 10));  
  
List<Personne> personnes = new List<Personne>();  
personnes.Add(Nicolas);  
personnes.Add(new Collaborateur("Jean", "Graillon", new DateTime(2020, 12, 10)));
```

Il est possible de créer un **objet Personne** avec une **instance de type Collaborateur**. Le type parent peut **utiliser des instances de types enfants** (et petits enfants...).

Ce mécanisme ne fonctionne pas dans l'autre sens, un enfant ne peut pas prendre une instance d'un type parent.

Héritage - classe abstraite

```
public abstract class Personne
{
    private string nom;

    private string prenom;

    public Personne(string nom, string prenom)
    {
        this.nom = nom;
        this.prenom = prenom;
    }
}
```

Une classe abstraite ne peut pas être instanciée.

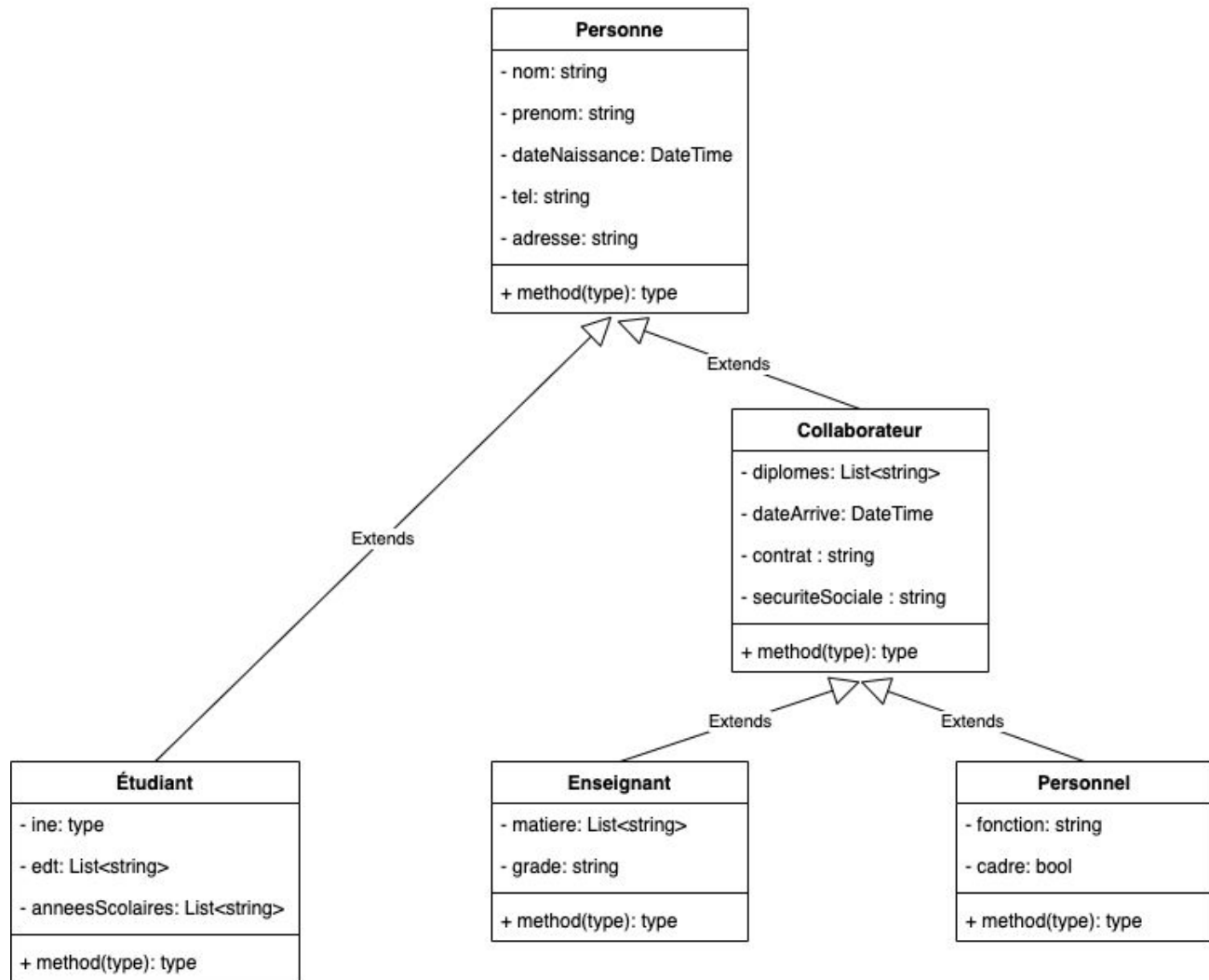
Héritage - classe abstraite

```
Personne Nicolas = new Personne("Nicolas", "Pascal", new DateTime(2020, 12, 10));  
  
List<Personne> personnes = new List<Personne>();  
personnes.Add(Nicolas);  
personnes.Add(new Collaborateur("Jean", "Graillon", new DateTime(2020, 12, 10)));
```

On **ne peut pas** instancier `Personne`, mais on peut toujours instancier **avec le type enfant** `Collaborateur` pour une `Personne`.

Héritage

Quelle classe
doit être
abstraite ?



Classe et méthode statique

Classe statique

On peut créer une **classe statique** avec le mot clé “static”.

Les classes statiques **ne peuvent pas être instanciées**, elles n’ont donc pas de constructeur

```
public static class UtilBien
{
    // Cette classe n'a pas de constructeur
}
```

Classe statique

Impossible d'instancier une classe statique.

```
UtilBien utilBien = new UtilBien();
```

class LSD_SIO2_projet1.UtilBien

CS0723: Impossible de déclarer une variable de type static 'UtilBien'

Classe statique

Ajout d'une méthode.

Toutes **les méthodes d'une classe statique sont obligatoirement statiques**. Il est nécessaire d'ajouter le mot-clé à chaque méthode.

```
public static class UtilBien
{
    // Cette méthode permet de renouveler un bail
    public static void renouvelerBail(Bien bien)
    {
        Bail dernierBail = bien.getBails().Last();
        DateTime dateFin = dernierBail.DateFin;
        dateFin = dateFin.AddYears(3);
        dernierBail.DateFin = dateFin;
    }
}
```

Classe statique

Pour appeler une méthode statique :

```
UtilBien.renouvelerBail(appartement);
```

Console.WriteLine est une méthode statique.

Méthode statique

Il est possible d'avoir **une méthode statique dans une classe non statique**.

```
public class UtilBien
{
    public UtilBien()
    {
    }

    public static void renouvelerBail(Bien bien)
    {
        Bail dernierBail = bien.getBails().Last();
        DateTime dateFin = dernierBail.DateFin;
        dateFin = dateFin.AddYears(3);
        dernierBail.DateFin = dateFin;
    }
}
```

Classe statique

- Une classe qui **ne peut pas** être instanciée
- Permet de stocker proprement des méthodes utilitaires
- Permet de stocker proprement des variables globales
- Il est possible d'avoir une méthode statique dans une classe non statique.

Le [Design Pattern Singleton](#) utilise les méthodes statiques.

Interface

Interface

Une interface définit un contrat. Chaque classe ou structure qui implémente une interface doit implémenter ses membres.

Plus d'infos sur la bible [ici](#).

Le [Observer Design Pattern](#) utilise les interfaces.

Interface - exemple

Création d'une interface

```
public interface IDescription
{
    // Propriété
    1 référence
    List<string> lignesDescription { get; set; }

    // Déclaration d'une méthode sans corps
    0 références
    public void ajouterDescription();

    // Déclaration d'une méthode avec corps
    0 références
    public void afficherDescription()
    {
        foreach (String ligne in lignesDescription)
        {
            Console.WriteLine(ligne);
        }
    }
}
```

Il est possible de déclarer une méthode sans corps.

Interface - implémentation

Les membres de l'interface **sont implémentés dans la classe**. C'est obligatoire.

```
public class Recette : IDescription
{
    public List<string> lignesDescription { get => throw new NotImplementedException(); set => t

    public void ajouterDescription()
    {
        throw new NotImplementedException();
    }

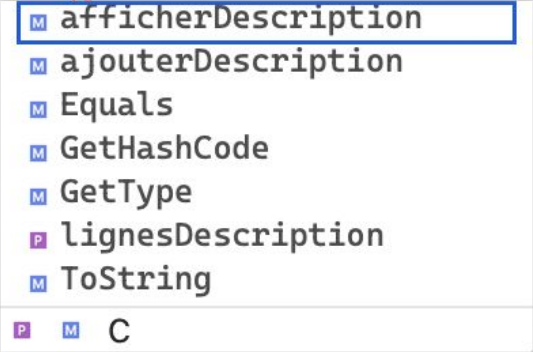
    public void afficherDescription()
    {
        throw new NotImplementedException();
    }
}
```

Interface - utilisation

Les interfaces peuvent être utilisées pour itérer sur des objets après avoir été castés. Seules les méthodes de l'interface sont utilisables après transformation.

```
List<IDescription> descriptions = new List<IDescription>();  
descriptions.Add((IDescription)nicolas); // On peut caster Collaborateur en IDescription
```

```
foreach(IDescription desc in descriptions)  
{  
    desc.afficherDescription();  
    desc.  
}
```



A dropdown menu showing the available methods for the `IDescription` interface. The methods are listed with their access modifiers: `M` for methods and `P` for properties. The first method, `afficherDescription`, is highlighted with a blue border.

- `M` afficherDescription
- `M` ajouterDescription
- `M` Equals
- `M` GetHashCode
- `M` GetType
- `P` lignesDescription
- `M` ToString

`P` `M` `C`

Interface - résumé

- L'interface est un contrat avec les classes qui l'implémentent
- Tous les membres de l'interface doivent être implémentés
- Il est possible de prédéfinir un corps dans une méthode
- L'héritage s'applique aux interfaces

Le design pattern observer ([Observer](#)) utilise les interfaces.

Redéfinition de méthode

(Polymorphisme)

Redéfinition de méthode - parent

```
// Méthode sans corps, DOIT être surchargée chez les enfants
// Peut être utilisé dans une classe abstraite
public abstract string getDesignation();

// Méthode avec corps, PEUT être surchargée par les enfants
// Peut être utilisé dans tout type de classe
public virtual string getFullDesignation()
{
    return this.nom + " " + this.prenom;
}
```

Redéfinition une méthode - enfant

```
// Surcharge de Abstract
// La méthode est obligatoire
public override string getDesignation()
{
    return this.nom + " " + this.prenom + " " + this.dateArrive.ToString();
}

// Surcharge de virtual
// La méthode est facultative
public override string getFullDesignation()
{
    return base.getFullDesignation() + " " + this.dateArrive.ToString();
}
```